DevConnect

A Full-Stack Social API

3 Months of C# & .NET — Learning Showcase

Presented by: *[Your Name]*

July 14, 2026

Agenda

1. What is DevConnect?
2. Tech Stack
3. Architecture Overview
4. Live Demo
5. Deep Dive — Key Concepts
6. Security
7. Quality: Testing, Logging, Caching
8. What I Learned & What's Next
9. Q&A

What is DevConnect?

A **social networking REST API** where users can:

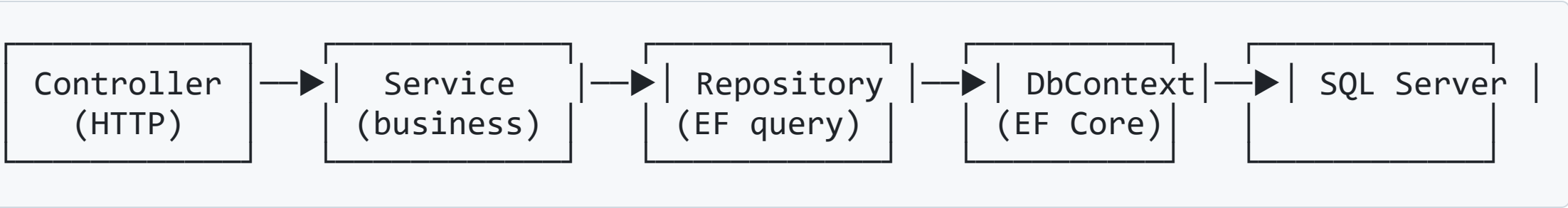
- Register / log in (JWT + Google/GitHub OAuth)
- Create, read, update, delete **posts**
- **Like** and **comment** on posts
- Browse posts with **pagination & sorting**

> Built to consolidate everything I learned in C# and .NET.

Tech Stack

Layer	Technology
Language	C# 12
Backend	ASP.NET Core Web API (.NET 8)
ORM / Data	Entity Framework Core 9 (Code-First)
Database	SQL Server
Auth	JWT Bearer + OAuth2 / OIDC (Google, GitHub)
Password	BCrypt hashing
Mapping / Validation	AutoMapper + FluentValidation
Performance	Output Caching (tag-based invalidation)
API Docs	Swagger / OpenAPI

Architecture Overview



- **Controllers** — handle HTTP, return status codes
- **Services** — business logic, mapping, ownership rules
- **Repositories** — EF Core queries only
- **DTOs** — decouple API from DB models
- **DI** — everything wired in `Program.cs`

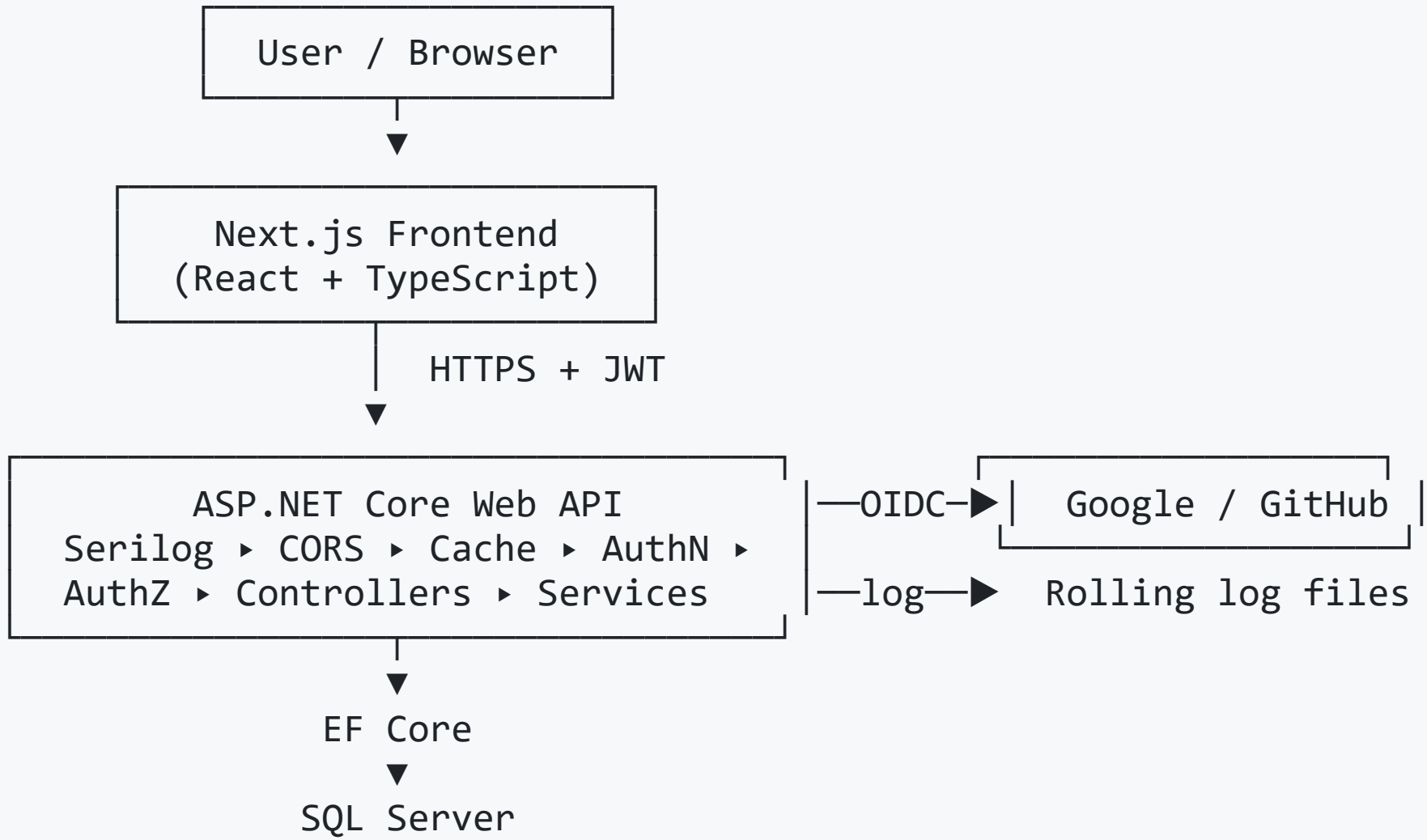
> Note: full chain is used by **Posts**; Auth/Users/Comments/Likes use `DbContext` directly.

Why This Architecture?

- **Separation of concerns** — each layer one job
- **Testable** — mock the repository, test the service
- **Maintainable** — change DB logic without touching controllers
- **SOLID** — depends on interfaces, not concrete classes

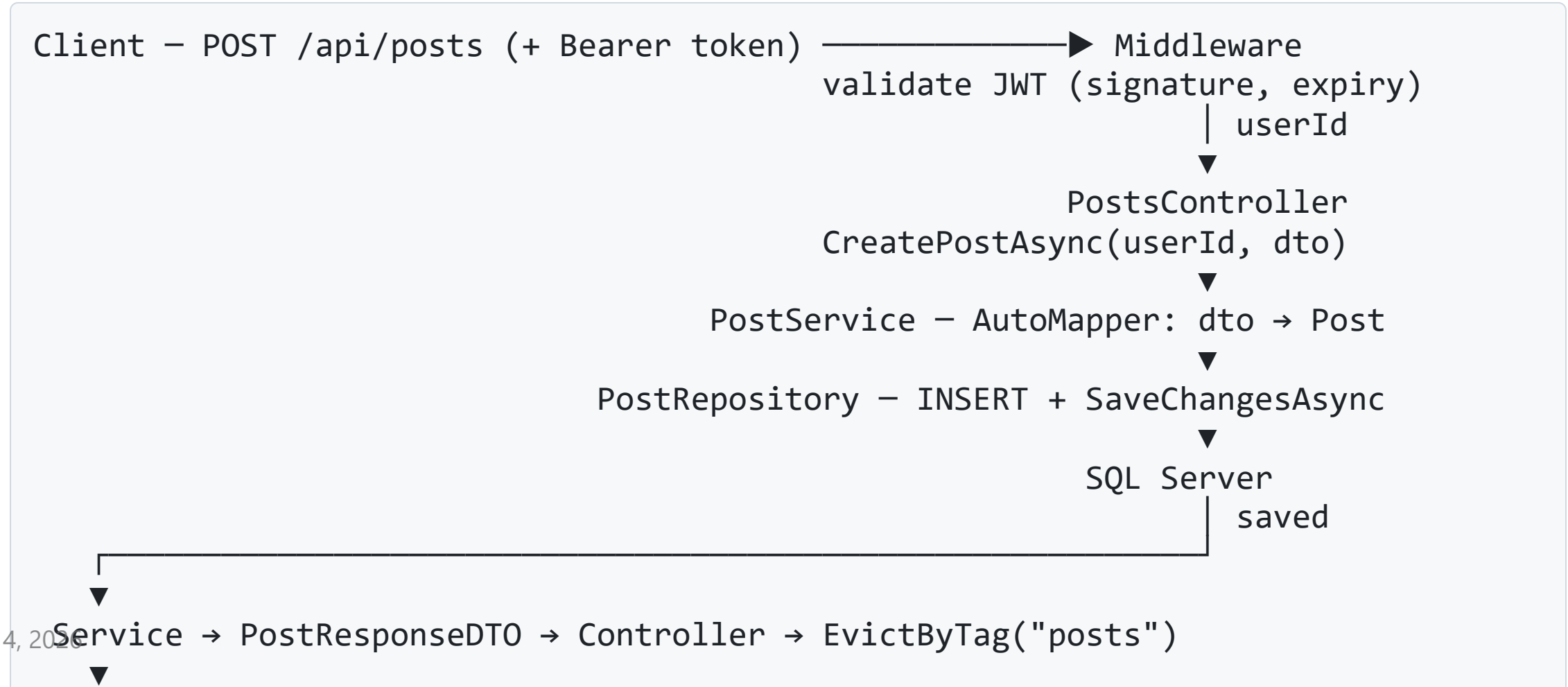
> "Thin controllers, smart services, dumb repositories."

System Design



Post API — Request Flow

Example: `POST /api/posts` (create a post)



Post API — Read Path (cached + paged)

`GET /api/posts?pageNumber=1&pageSize=10&sortBy=likes`

1. **Output cache** checked first — hit → return instantly
2. Miss → `PostsController.GetAll` → `PostService.GetPagedPostsAsync`
3. Service **clamps** page bounds (`PageSize` 1–100)
4. Repository builds `IQueryable` : `Include` → **sort** → `Skip` / `Take`
5. Returns `PagedResult<T>` with `totalCount` , `totalPages`
6. Response cached for 30s under tag `posts`

> Writes (create/update/delete) **evict** the `posts` tag → no stale reads.

Live Demo

Register → Login → Create Post → Like & Comment
→ Pagination → Swagger → Logs

Deep Dive: Dependency Injection

```
builder.Services.AddScoped<IPostRepository, PostRepository>();
builder.Services.AddScoped<IPostService, PostService>();
builder.Services.AddScoped<IAuthService, AuthService>();
```

- **Scoped** = one instance per HTTP request
- Constructors receive dependencies automatically
- Enables mocking in unit tests

Deep Dive: DTOs & AutoMapper

Problem: never expose `PasswordHash` or internal fields.

```
CreateMap<Post, PostResponseDTO>()
    .ForMember(d => d.AuthorName,
               o => o.MapFrom(s => s.User.Username));
```

- **DTOs** shape exactly what the client needs
- **AutoMapper** removes repetitive mapping code

Deep Dive: EF Core

- Models → tables via **navigation properties**
- LINQ → **SQL** (translated, deferred execution)
- **Migrations** version the schema
- **Pagination** with `Skip` / `Take`

```
var posts = await q
    .Skip((page - 1) * size)
    .Take(size)
    .ToListAsync();
```

Security: Authentication

JWT Flow:

```
1. Login → verify password (BCrypt)
2. Issue signed JWT with claims (id, email, role)
3. Client sends: Authorization: Bearer <token>
4. Middleware validates signature + expiry
```

- Passwords **hashed** with BCrypt (never plaintext)
- Also supports **Google & GitHub** OAuth login

Security: Authorization

- **Role-based:** `[Authorize(Roles = "Admin")]`
- **Ownership checks** in the service layer

```
if (post.UserId != userId && role != "Admin")
    return false; // can't edit others' posts
```

- **CORS** allows only trusted frontend origins

Quality: Cross-Cutting Concerns

Feature	Purpose
Serilog	Structured, queryable logs
Output Caching	Faster read-heavy endpoints
Pagination & Sorting	Scalable list responses
FluentValidation	Clean, reusable input rules
Swagger	Interactive API docs

Quality: Testing

- **Unit tests** — service logic with **Moq** (isolated)
- **Integration tests** — repository against a test DB
- Framework: **NUnit** (Arrange–Act–Assert)

```
dotnet test   # all green ✅
```

> Focused on the service layer — where business rules live.

What I Learned

- Building a clean, layered .NET Web API end-to-end
- Real-world **auth** (JWT + OAuth/OIDC)
- **EF Core** modeling, migrations, query performance
- Writing **testable** code with DI and interfaces
- Connecting a **React/Next.js** frontend to the API

What's Next

- Refresh tokens for longer sessions
- Global exception-handling middleware
- Rate limiting
- Higher test coverage
- Secrets in a vault (Azure Key Vault)
- Caching & indexing for scale

Thank You!

Questions?

DevConnect — C# / .NET Learning Showcase